NETWORKING & CYBERSECURITY

Firewalls, NAT & Proxies

Consolidated study notes covering firewall fundamentals, network address translation, and forward & reverse proxy architecture

Firewalls NAT Proxies NGINX

Table of Contents

SECTION 1

Firewalls

1. What is a Firewall?

A firewall is a hardware or software security system that monitors and controls network traffic between communication points according to predefined security rules.

Its primary job is to decide one of three outcomes:

- **Allow** the traffic
- **Reject** the traffic
- **Drop** the traffic

> **KEY CONCEPT**
> A firewall is not defined by separating public and private networks. It is defined by enforcing security rules at a network boundary.

2. Where Can Firewalls Be Placed?

A firewall can be placed anywhere traffic needs to be controlled. Possible deployments include:

- Public ↔ Private *(most common)*
- Public ↔ Public
- Private ↔ Private
- On a single host (Host Firewall)

The key idea: a firewall enforces security rules wherever there is a network boundary.

3. Public vs Private Networks

A **public network** uses publicly routable IP addresses. A **private network** uses private IP ranges:

- `10.0.0.0/8`
- `172.16.0.0/12`
- `192.168.0.0/16`

Private IP addresses cannot be routed directly over the Internet.

4. Why Public ↔ Private is the Most Common Deployment

Modern organizations usually keep their internal servers on private IP addresses. Example topology:

```
Internet (Public)
    |
Public IP
    |
Firewall + NAT
    |
Private Network
    |
Web Server
```

The firewall receives traffic destined for the public IP, applies its rules, performs NAT, and forwards traffic to the server's private IP — this is why Public ↔ Private is the most common deployment today.

5. Public ↔ Public Firewalls Also Exist

A server may have a public IP directly:

```
Internet
   |
Firewall
   |
Web Server (Public IP)
```

Both sides use publicly routable addresses. This architecture is common with cloud VMs, dedicated servers, and simpler hosting environments. Firewalls are therefore not limited to Public ↔ Private deployments.

6. Allow vs Reject vs Drop

Allow

The firewall forwards the packet: `Client → Firewall → Server`. The destination receives the packet normally.

Reject

The firewall blocks the packet and immediately notifies the sender — for TCP this is most commonly a `TCP RST`. The client instantly knows the connection was refused.

Drop

The firewall silently discards the packet (`Client → Firewall → (no response)`). The sender waits until a timeout occurs.

7. TCP RST

RST is **not** a firewall-specific flag — it is the standard TCP mechanism for immediately refusing or terminating a connection. Both scenarios below commonly use RST:

Scenario	Flow	Meaning
Closed Port	Client → Server → RST	No application is listening
Firewall Reject	Client → Firewall → RST	Firewall refuses before forwarding

Both a closed port and a rejecting firewall commonly send TCP RST packets.

8. Can We Tell Whether the RST Came From the Firewall?

Normally, yes — check the source IP of the RST:

- RST Source = Firewall IP → the firewall generated the RST
- RST Source = Target Server IP → likely the server rejected the connection (closed port)

> **Transparent Firewalls**
>
> Some firewalls generate the RST using the server's IP address (a forged source IP). In this case, a closed port and a transparent firewall reject look identical — from that packet alone it is often impossible to determine which generated the RST.

9. Stateless Firewall

A stateless firewall has no memory and treats every packet independently, checking only the current packet against its rules. It does not remember previous packets or active connections — so administrators often need separate rules for outgoing and incoming traffic.

10. Stateful Firewall

A stateful firewall keeps a state table and tracks active network connections, using connection state when making decisions. Example:

```
Client → SYN
Firewall records connection
Server → SYN,ACK
Firewall recognizes: "belongs to existing connection"
```

Return traffic is automatically allowed because the firewall remembers the connection.

11. Does a Stateful Firewall Detect Attacks?

Not by definition. The defining feature of a stateful firewall is that **it tracks connection state** — its job is not to identify attackers. However, because it knows connection state, it can reject packets that don't belong to any legitimate connection. Advanced features like IDS, IPS, DPI, and Next-Generation Firewall capabilities are additional features, not what makes a firewall "stateful."

12. Why Stateful Firewalls Improve Security

A stateful firewall can determine whether a packet belongs to an existing connection. For example, a random ACK packet arrives, the firewall checks its state table, finds no matching connection, and drops the packet. A stateless firewall only evaluates the packet against static rules and has no connection history.

13. How Firewalls Affect Nmap

Firewall Action	Nmap Sends	Response	Nmap Reports
Allows	SYN	SYN, ACK	**open**
Rejects	SYN	RST	**closed**
Drops	SYN	(no response)	**filtered**

14. What Does "Filtered" Mean?

A filtered port means Nmap did not receive enough information to determine whether the port is open or closed. The most common reason is that a firewall silently dropped the packets — but other possibilities exist too: router filtering, ACL filtering, packet loss, or general network problems.

> **IMPORTANT**
> Nmap cannot prove that a firewall blocked the traffic; it only knows that the traffic was filtered somewhere.

Key Takeaways

- A firewall enforces security rules at a network boundary.
- Firewalls can exist between Public↔Private, Public↔Public, Private↔Private, or on a single host.
- Public↔Private is the most common deployment today because organizations commonly use NAT.

- TCP RST is commonly used by both closed ports and rejecting firewalls.
- A transparent firewall forging the server's IP may make its RST indistinguishable from the server's own.
- Stateless firewalls examine each packet independently.
- Stateful firewalls remember active connections using a state table.
- Stateful means connection-aware, not attack-aware.
- Nmap reports "filtered" when it cannot determine whether a port is open or closed due to insufficient responses.

Addendum: IPv6 and NAT — Is NAT Still Required?

Question: if both the client and the web server have IPv6 addresses, it's most likely a public IPv6 address — so would NAT be required?

Short answer: No. If both the client and the web server have globally routable IPv6 addresses, NAT is generally not required. This is one of the biggest architectural differences between IPv4 and IPv6.

IPv4

IPv4 has a limited address space (about 4.3 billion addresses), so the world ran out of public addresses. That is why organizations commonly rely on NAT:

```
Internet
   |
Public IP
   |
Firewall + NAT
   |
192.168.1.20 (Private)
```

NAT became almost essential in the IPv4 world.

IPv6

IPv6 has an enormous address space (2^{128} addresses). Every device can have its own globally unique public IPv6 address:

```
Client (2001:db8:1::10)
   |
Internet
   |
Firewall
   |
Web Server (2001:db8:2::20)
```

The client can route directly to the server — no NAT is needed.

Do Firewalls Disappear With IPv6?

No. The firewall remains extremely important. It still decides things like:

- Allow HTTPS (443)
- Block SSH (22)
- Allow ICMPv6
- Block unwanted inbound connections

The topology becomes a **Public ↔ Public** deployment:

```
Internet (Public IPv6)
   |
Firewall
   |
Web Server (Public IPv6)
```

Is NAT Used With IPv6?

Generally, no — one of IPv6's core design goals was to eliminate the need for NAT. Forms of IPv6 NAT (such as NAT66) exist but are uncommon and not the normal deployment model.

> **CONCLUSION**
>
> **IPv4 world:** Public↔Private with NAT is the most common firewall deployment.
> **IPv6 world:** As IPv6 adoption grows, Public↔Public without NAT becomes much more common, since every device can have its own globally routable address. The "public↔public" intuition becomes far more accurate in a native IPv6 environment than in today's predominantly IPv4-based deployments.

SECTION 2

NAT — Network Address Translation

General Definition of NAT

- NAT is a **networking function/process**, not a physical device.
- It is usually performed by a router, firewall, or dedicated gateway.

> **DEFINITION**
> NAT is the process of translating network addresses in packets as they pass through a network device.

This definition describes *what* NAT does, not *why* a particular type of NAT exists.

Traditional NAT (The NAT Most People Mean)

When networking courses, cybersecurity courses, or interviews simply say "NAT," they are almost always referring to **traditional IPv4 NAT/PAT**.

Purpose

- Allow private IPv4 devices to communicate with the public Internet.
- Conserve the limited number of public IPv4 addresses by letting many internal devices share one public IPv4 address.

Translation

Traditional NAT translates Private IPv4 → Public IPv4 (and back for return traffic). The most common implementation is **PAT (Port Address Translation)**, which also translates source port numbers.

NAT Family

NAT is a general category. Examples include:

Type	Purpose	Translation
Traditional NAT/PAT	IPv4 address conservation	Private IPv4 ↔ Public IPv4
NAT64	Enable IPv4-only ↔ IPv6-only communication	IPv4 ↔ IPv6

Technically, NAT64 is a type of NAT — but when someone simply says "NAT," they almost always mean traditional IPv4 NAT/PAT, unless the discussion is specifically about IPv6.

General Definition vs Purpose

The general NAT definition describes *what* NAT does (address translation). Each NAT type has its own purpose:

- Traditional NAT → solve IPv4 public address exhaustion
- NAT64 → enable IPv4 and IPv6 interoperability

Why Private IPv4 Addresses Cannot Be Routed on the Internet

Private IPv4 addresses are not globally unique — millions of homes can simultaneously have `192.168.1.10`. If Internet routers accepted private IPs, they would not know which network a packet destined for that address should reach.

Therefore, Internet routers do not advertise routes for private address ranges, and packets with private destination addresses are discarded on the public Internet. This is a global routing policy because private addresses are intentionally reusable.

Why NAT Is Needed

When a device with a private IPv4 address wants to access the Internet, the router replaces the private source IP with its public IP and stores the translation in a NAT table. When the reply returns, the router reverses the translation and forwards the packet to the correct internal device.

Sharing One Public IPv4 Address

Traditional home routers typically use PAT (Port Address Translation), which translates the source IP address and source port (when necessary) — allowing many simultaneous connections to share one public IPv4 address.

Important Clarification About Port Numbers

There are two separate questions here:

1. How does the router identify the destination device?

The router identifies the internal device using its **private IP address**. For identifying which device on the LAN should receive the packet, the private IP is sufficient.

2. Why are port numbers still needed?

Port numbers are not primarily used because the router cannot distinguish devices. They are needed because NAT must uniquely identify **communication sessions (connections)** while many connections share a single public IPv4 address.

SUMMARY

Private IP → identifies the internal device.
Port number → identifies the specific connection/application on that device.
These are different purposes and should not be confused.

Key Takeaways

- NAT is a process, not hardware.
- The general definition of NAT is address translation.
- Traditional NAT exists primarily to conserve public IPv4 addresses.
- NAT64 is also a type of NAT but serves a different purpose (IPv4/IPv6 communication).
- Private IPv4 addresses are not routed on the public Internet because they are not globally unique.
- The router uses the private IP to identify the destination device on the LAN.
- Port numbers distinguish individual communication sessions, allowing many simultaneous connections to share a single public IPv4 address.

SECTION 3

Proxy & Reverse Proxy

Proxy (Forward Proxy)

A **forward proxy** acts on behalf of the client.

```
Client → Forward Proxy → Server
```

The client sends requests to the proxy, and the proxy forwards them to the destination server. The destination server sees the proxy's IP rather than the client's IP.

Common Uses

- Hide the client's IP address
- Access restricted websites
- Monitor internet usage
- Filter websites
- Cache frequently accessed content

Real-World Examples

- School or college internet filtering
- Corporate internet access
- VPNs (similar concept, though VPNs operate at a broader network level)

> **KEY IDEA**
> A forward proxy represents and protects the **client**.

Reverse Proxy

A **reverse proxy** acts on behalf of the server.

```
Client → Reverse Proxy → Backend Server
```

Clients believe they are communicating directly with the server, but every request first reaches the reverse proxy.

> **KEY IDEA**
> A reverse proxy represents and protects the **server**.

Main Difference

Forward Proxy	Reverse Proxy
Represents the client	Represents the server
Controlled by the client / client's organization	Controlled by the server owner
Hides the client	Hides the backend server
Used when clients access the internet	Used when servers provide services to clients

The fundamental difference is **who the proxy represents**, not how packets are forwarded.

Why Use a Reverse Proxy?

A reverse proxy provides a central layer that handles networking and security tasks before requests reach the backend servers. Common responsibilities include:

- Request forwarding
- Load balancing
- SSL/TLS termination
- Caching
- Rate limiting
- Request filtering
- Hiding backend servers

How a Reverse Proxy Helps Against DDoS

A reverse proxy does not magically stop DDoS attacks simply by existing. Instead, it provides a layer where protection mechanisms can be applied before traffic reaches the backend server. It can:

- Drop malicious requests
- Rate limit abusive clients
- Filter attack traffic
- Absorb attacks using large distributed infrastructure (e.g., Cloudflare)
- Hide the real backend server's IP address

The backend server receives only the requests that the reverse proxy decides to forward.

Why Not Let the Web Server Do Everything?

A normal web server *can* perform many of the same functions. However, using a reverse proxy separates responsibilities.

Without a reverse proxy:

```
Internet → Web Server
```

The web server must serve content, handle HTTPS, filter attacks, cache responses, load balance, and process application logic — all at once.

With a reverse proxy:

```
Internet → Reverse Proxy → Backend Server
```

The reverse proxy handles networking and security tasks, while the backend server focuses purely on application logic.

Benefits

- Simpler backend servers
- Centralized configuration
- Easier maintenance
- Better scalability
- Improved performance

This is called **separation of responsibilities** (or specialization).

NGINX

NGINX (pronounced "Engine-X") is one of the world's most popular web servers, reverse proxies, and load balancers. As a reverse proxy, NGINX sits in front of one or more backend servers:

```
Client → NGINX → Backend Server(s)
```

NGINX can forward requests, load balance, terminate HTTPS (SSL/TLS), cache responses, serve static files, and route requests to different backend services.

Is NGINX Famous?

Yes — it is one of the most widely used reverse proxies on the Internet. During reconnaissance in ethical hacking, you will often encounter the header `Server: nginx`, meaning NGINX is handling the incoming HTTP traffic. It may be serving the website directly, acting as a reverse proxy, or both.

Other Reverse Proxies

Examples include NGINX, HAProxy, Apache HTTP Server, Traefik, and Caddy. All perform the same fundamental role: receive client requests, represent the server, forward requests to backend servers,

and return responses to clients. They mainly differ in performance, features, configuration, and specialization.

Key Takeaways

- A forward proxy represents the client.
- A reverse proxy represents the server.
- Both are intermediary servers; the difference is who they work for.
- Reverse proxies centralize networking, security, and traffic management.
- Backend servers focus on application logic.
- NGINX is one of the most common reverse proxies used in modern web infrastructure.